

Introduction to Algorithms

Levon Prazyan

Course Overview

- Introduction
- Sorting and Searching
- Arrays and Lists
- Stacks and Queues, Double Ended Queue
- Heaps
- Trees
- Hash Tables
- Dynamic Programming
- Greedy Algorithms
- Graph Algorithms
- Network Flow
- NP-Completeness

Graph: Definition

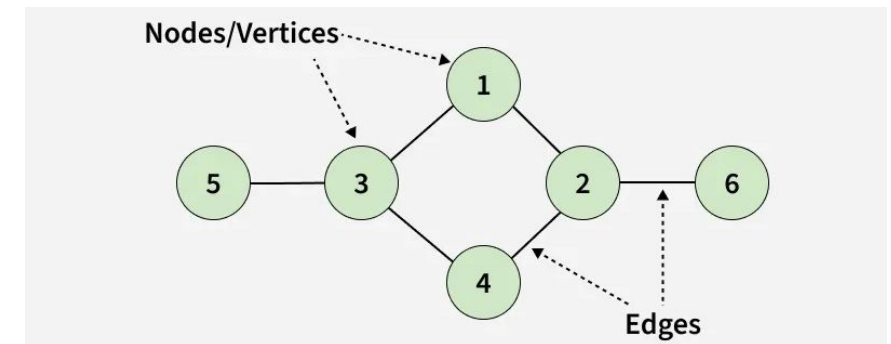
A graph is a structure used to model relationships between objects.

A graph is defined as a pair

- $G = (V, E)$
where:
- V is a set of **vertices** (also called **nodes**) — the objects
- E is a set of **edges** — the connections between those objects

Searching a graph means systematically following the edges of the graph to visit the vertices of the graph

$$E = \{(A, B), (B, C)\} \quad V = \{A, B, C\}$$



Type of Graphs

- **Undirected graph**

- Edges have no direction
- Example: friendship (mutual)

- **Directed graph (digraph)**

- Edges have direction
- Example: Twitter follows ($A \rightarrow B$)

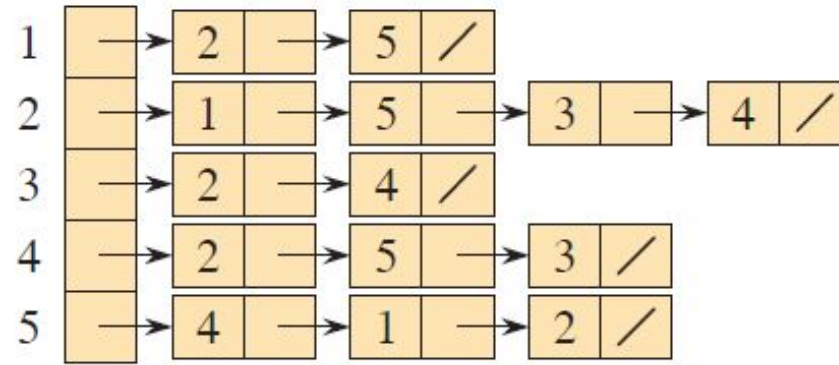
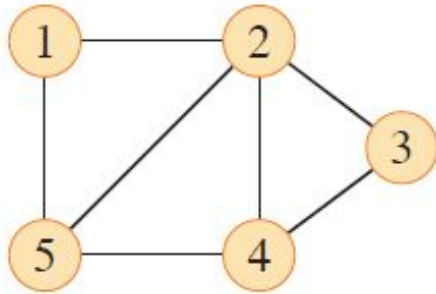
- **Weighted graph**

- Each edge has a weight (cost, distance, etc.)

- **Unweighted graph**

- All edges are equal

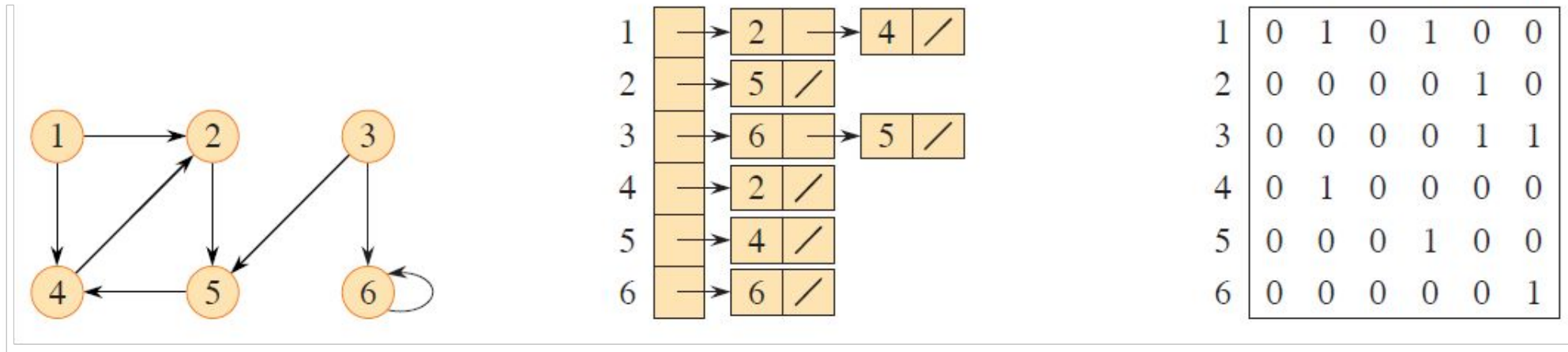
Representations of Graph (1)



	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

- Adjacency list
- Adjacency matrix

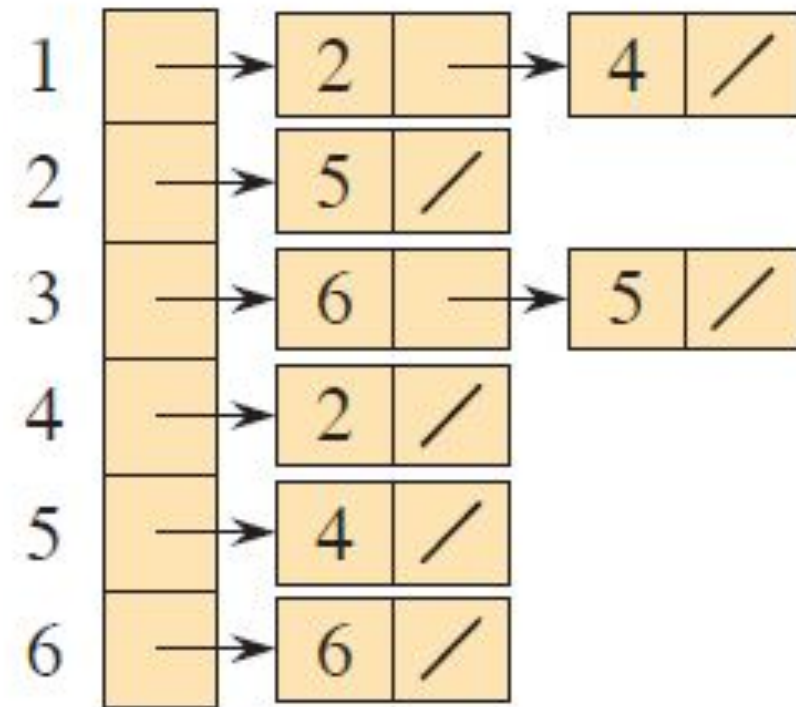
Representations of Graph (2)



- Choose adjacency list for sparse graphs.
 - less memory
- Choose adjacency matrix for dense graphs
 - quick checks for edges

Adjacency List Representation

- Graph $G = (V, E)$ consists of array Adj of $|V|$ lists.
- For each $v \in V$ the $Adj[v]$ list contains all u nodes, such $(u, v) \in E$.
- For directed graphs: $\sum_{u \in V} Adj[u].size = |E|$
- For undirected graphs: $\sum_{u \in V} Adj[u].size = 2 * |E|$
- Amount of memory: $M = \theta(|V| + |E|)$
- Time to traverse all edges: $T = \theta(|V| + |E|)$
 - Often $|V| = O(E)$; $T = \theta(E)$
- For weighed edges: $w: E \rightarrow R ; w(u, v) ; (u, v) \in E$



Adjacency Matrix Representation

- $A = (a_{ij})$ adjacency matrix of graph G consists of $|V| \times |V|$ elements such

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

- Amount of memory: $M = \theta(V^2)$
- Time to traverse all edges: $T = \theta(V^2)$
- For undirected graphs $A = A^T$

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Breadth-first Search (BFS)

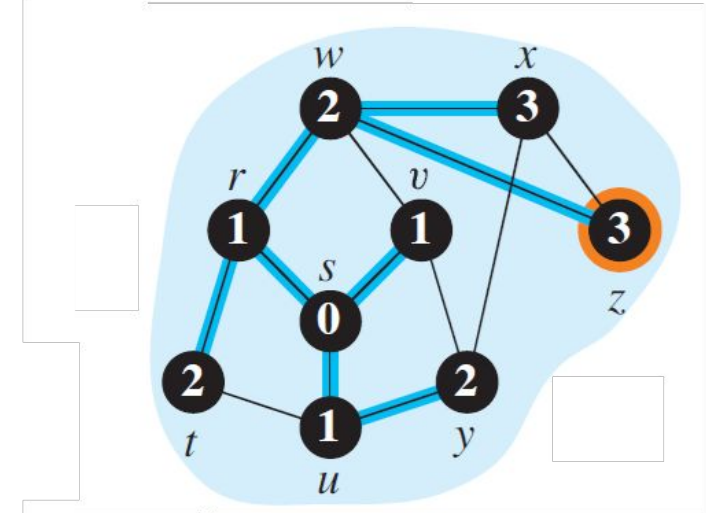
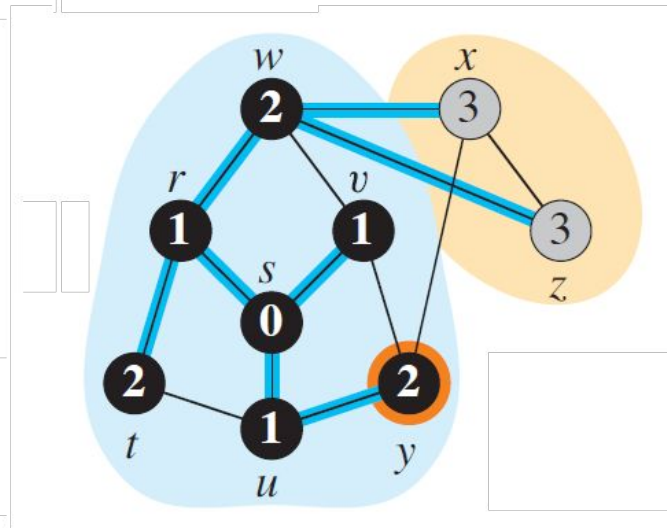
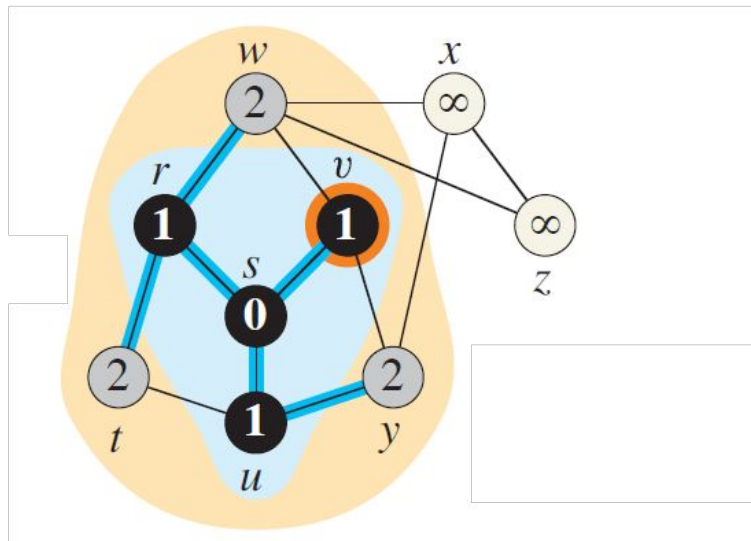
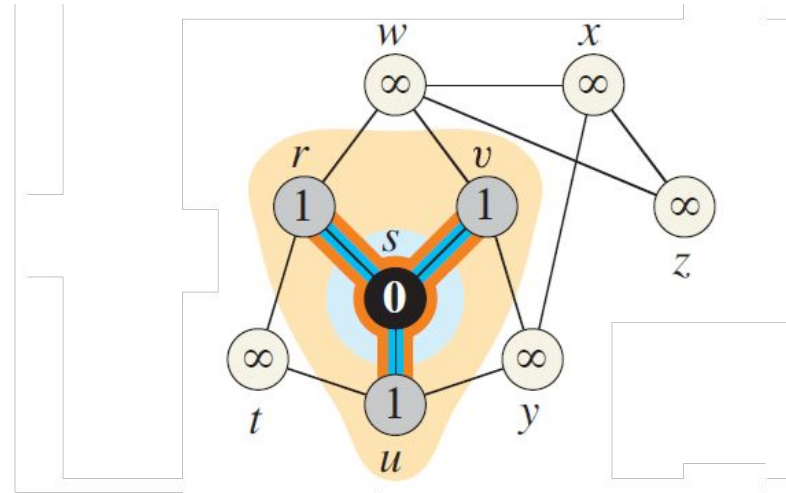
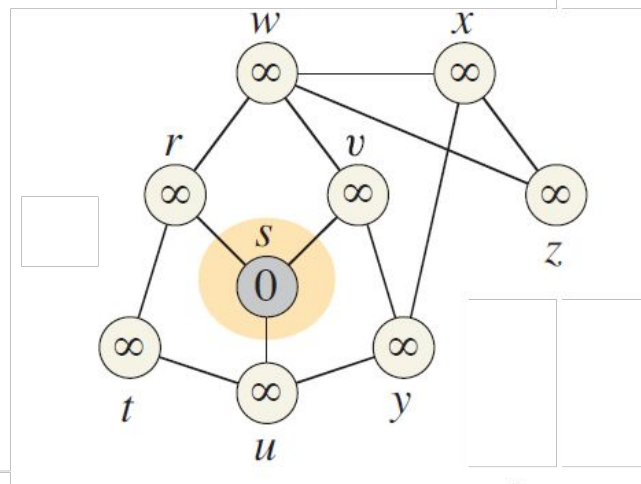
-

Given a graph $G = (V, E)$ and a distinguished **source** vertex s . Breadth-first search systematically explores the edges of G to “discover” every vertex that is reachable from s .

BFS computes the distance from s to each reachable vertex.

The distance to a vertex v equals the smallest number of edges needed to go from s to v .

BFS Steps



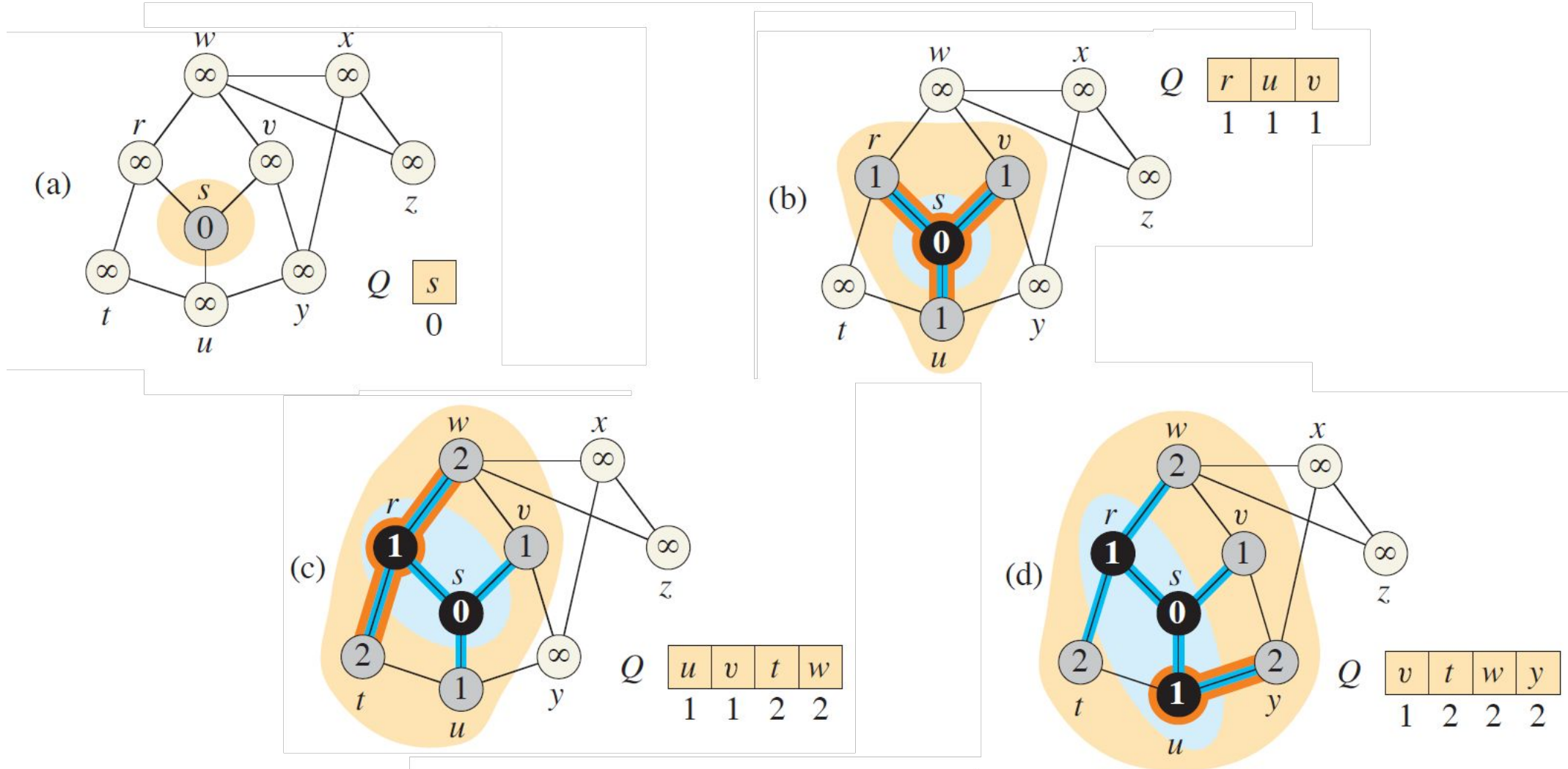
BFS

Pseudocode

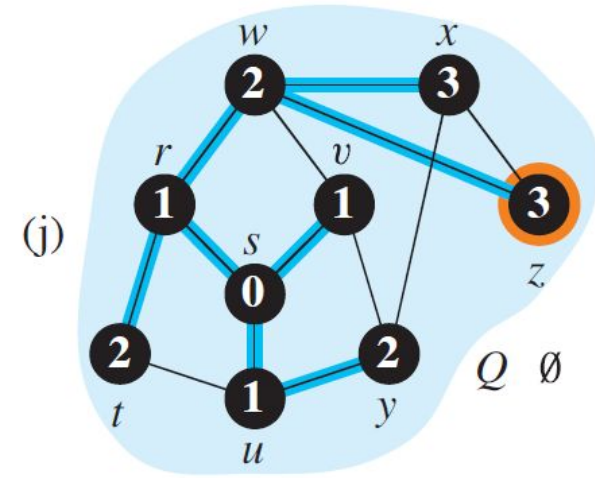
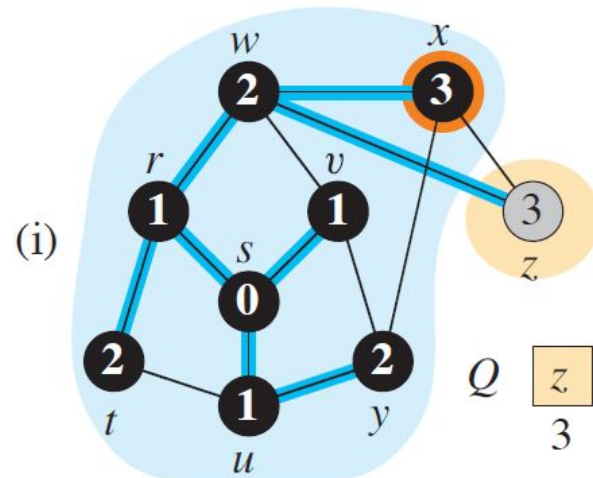
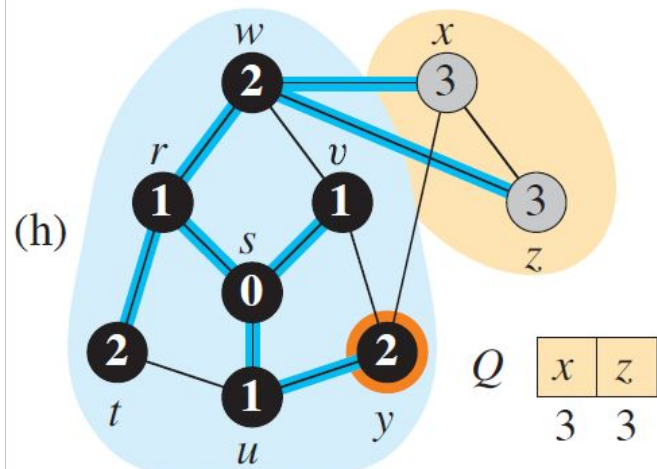
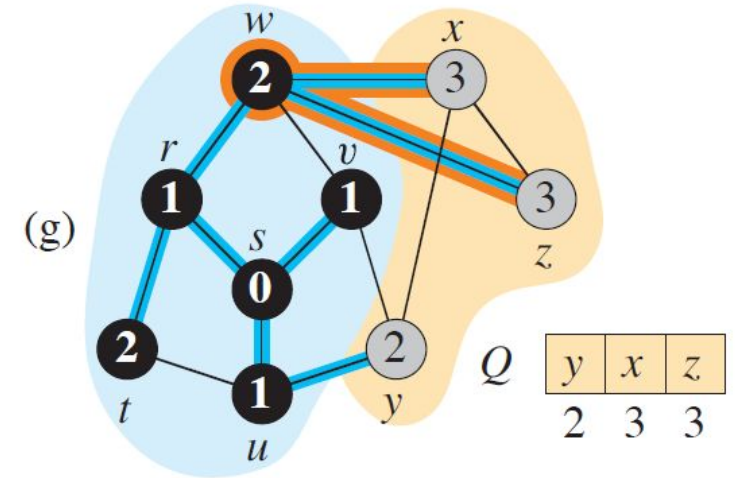
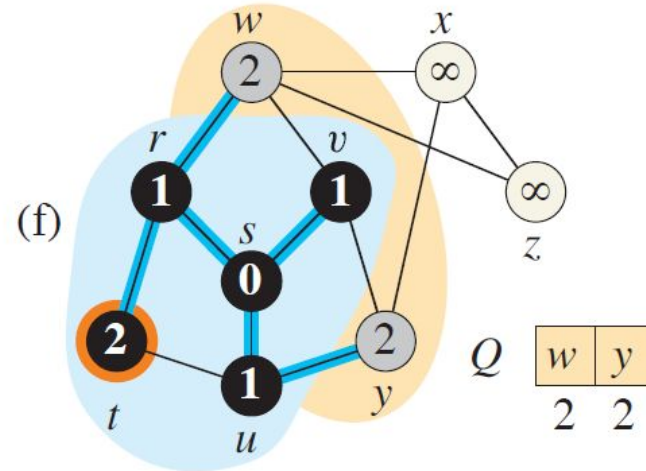
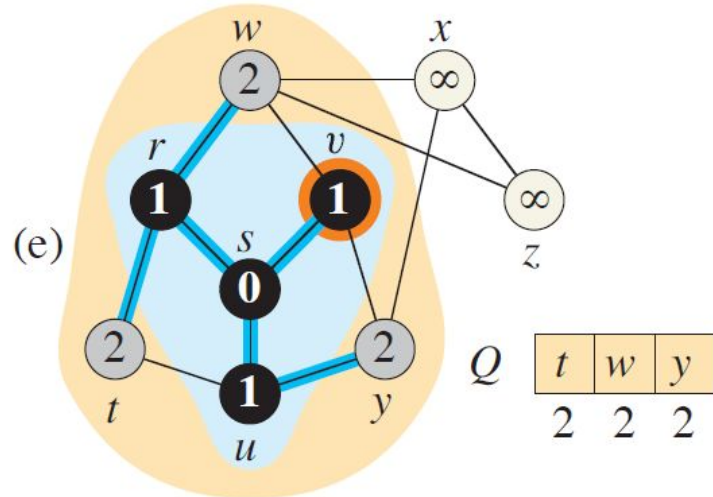
BFS(G, s)

```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each vertex  $v$  in  $G.Adj[u]$  // search the neighbors of  $u$ 
13         if  $v.color == \text{WHITE}$  // is  $v$  being discovered now?
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ ) //  $v$  is now on the frontier
18      $u.color = \text{BLACK}$  //  $u$  is now behind the frontier
```

BFS Example (1)



BFS Example (2)



BFS Analysis

```
BFS( $G, s$ )
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each vertex  $v$  in  $G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

- Q is a Queue data structure
 - *Enqueue* takes $O(1)$ time
 - *Dequeue* takes $O(1)$ time
 - All queue operations time $O(n)$
- An adjacency list is scanned when appropriate vertex is removed from Q .
 - Each edge is scanned only once.
- Sum of lengths of all $|V|$ lists is $\theta(E)$
- **Total time $O(V + E)$**

BFS: Shortest Path (1)

- Shortest path distance $\delta(s, v)$ from s to v is the minimum number of edges in any path from s to v .

Lemma 1: Let $G = (V, E)$ be a directed or undirected graph, let $s \in V$. Then for any $(u, v) \in E$ edge

$$\delta(s, v) \leq \delta(s, u) + 1$$

Proof: If u is reachable from s , then shortest path from s to v cannot be shorter than shortest path from s to u plus edge (u, v) .

If u is unreachable from s , then $\delta(s, u) = \infty$ and inequality holds.

BFS: Shortest Path (2)

Lemma 2: Let $G = (V, E)$ be a directed or undirected graph and let the BFS run on some $s \in V$. Then for any $v \in V$ vertex the $v.d \geq \delta(s, v)$ always.

Proof by induction:

- **Base:** $\delta(s, s) = 0 = s.d$ and $v.d = \infty \geq \delta(s, v)$ for $v \in V - \{s\}$
- **Induction step:** Let v be discovered from vertex u .
According to inductive hypothesis $u.d \geq \delta(s, u)$.
Then $v.d = u.d + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$.
 v is never visited again.
- **Termination:** All reachable vertices are visited.
For unreachable vertices $v.d = \infty \geq \delta(s, v)$

BFS: Shortest Path (3)

Lemma 3: Let $G = (V, E)$ be a directed or undirected graph and let at some point of BFS there are $(v_1, v_2, \dots, v_r)$ vertices in the Q queue. v_1 is the head of queue and v_r is the tail. Then $v_r.d \leq v_1.d + 1$ and $v_i \leq v_{i+1} ; i = \overline{1, r-1}$

Proof by induction:

- **Base:** Only s is in the Q : lemma holds.
- **Termination:** Q is empty: lemma holds
- **Induction step:**

Dequeuing: v_1 is removed. v_2 is the new head.

By hypothesis: $v_r.d \leq v_1.d + 1$ and $v_1.d \leq v_2.d$, so $v_r.d \leq v_2.d + 1$

Enqueuing: let v_{r+1} is added to the Q containing $(v_1, v_2, \dots, v_r)$.

If Q was empty, then the lemma holds: if not, assume u was removed and v_{r+1} is adjacent to u

By hypothesis: $u.d \leq v_1.d$ and $v_r.d \leq u.d + 1$

So: $v_r.d \leq u.d + 1 \leq v_1.d + 1 ; u.d + 1 = v_{r+1}.d$

$v_r.d \leq v_{r+1}.d \leq v_1.d + 1$

BFS: Correctness (1).

Theorem: Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$.

- Then, during its execution, BFS discovers every vertex $v \in V$ that is reachable from the source s , and upon termination, $v.d = \delta(s, v)$ for all $v \in V$.
- For any vertex $v \neq s$ that is reachable from s , one of the shortest paths from s to v is a shortest path from s to $v.\pi$ followed by edge $(v.\pi, v)$

BFS: Correctness (2).

Proof: Let's assume that for some $v \in V$; $v.d \neq \delta(s, v)$; ($v \neq s$).

By Lemma_2 $v.d \geq \delta(s, v) \rightarrow v.d > \delta(s, v)$

Let u be the vertex immediately preceding v on some shortest path from s to v :

$$\delta(s, v) = \delta(s, u) + 1 ; u.d = \delta(s, u)$$

$$u.d > \delta(s, v) = \delta(s, u) + 1 = \mathbf{u.d + 1}$$

A moment when u is removed from queue

1. $v.color$ is white : then v will $v.d = u.d + 1$
2. $v.color$ is black : then v is removed earlier, so $v.d < u.d$
3. $v.color$ is gray : then v has been added to queue when removing some w .
 $v.d = w.d + 1$ and $w.d \leq u.d$, so $v.d \leq u.d + 1$

Depth-first Search (DFS)

-

Given a graph $G = (V, E)$ and a distinguished **source** vertex s . Depth-first search explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it. Once all of v 's edges have been explored, the search “backtracks” to explore edges leaving the vertex from which v was discovered.

DFS Pseudocode

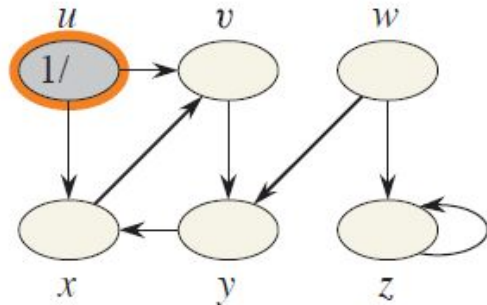
DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

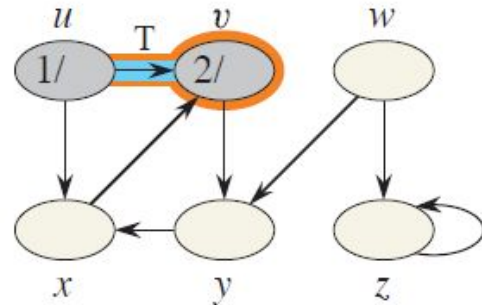
DFS-VISIT(G, u)

```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each vertex  $v$  in  $G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $time = time + 1$ 
9   $u.f = time$ 
10  $u.color = \text{BLACK}$ 
```

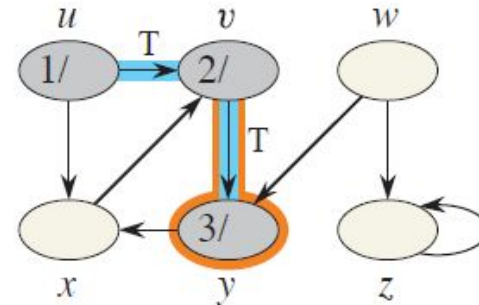
DFS Example (1)



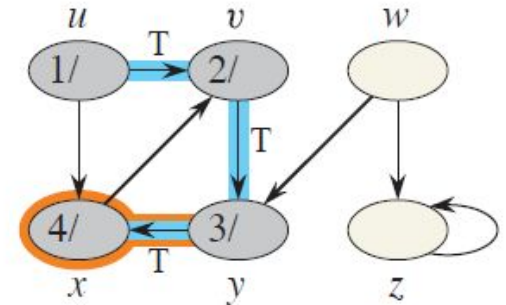
(a)



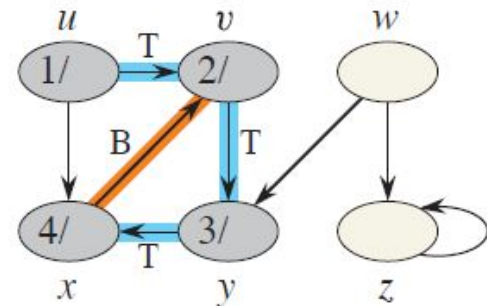
(b)



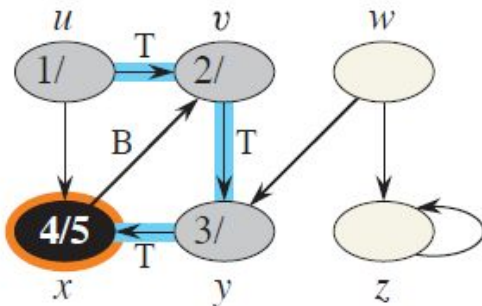
(c)



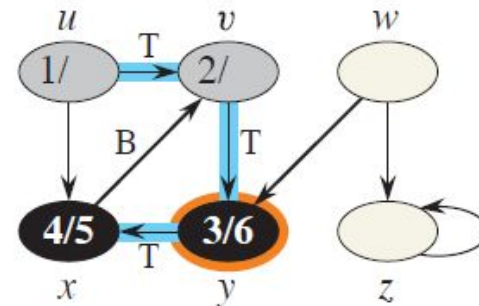
(d)



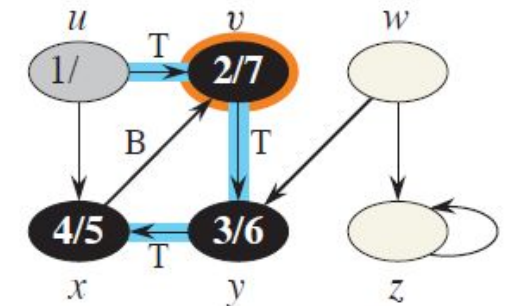
(e)



(f)

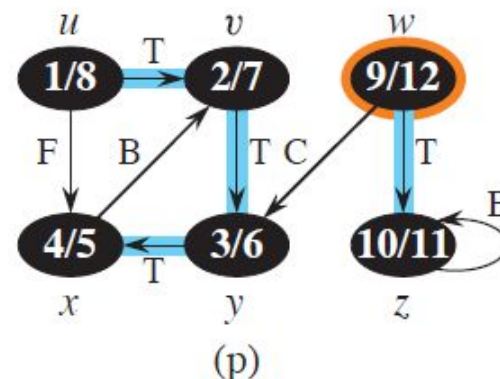
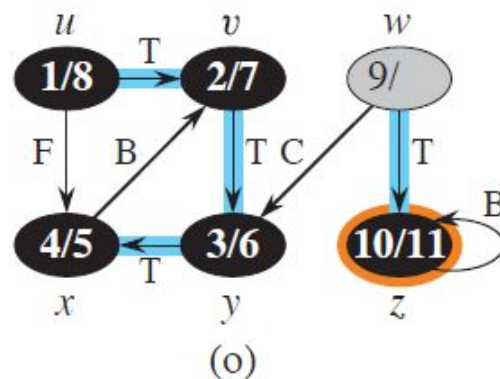
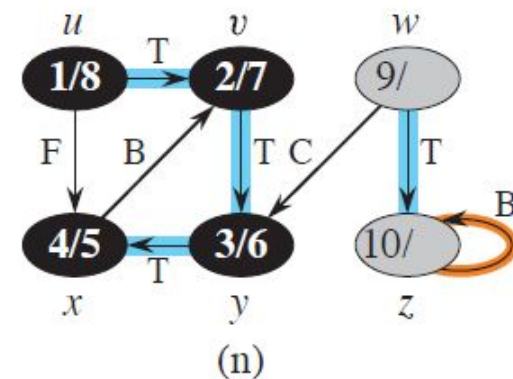
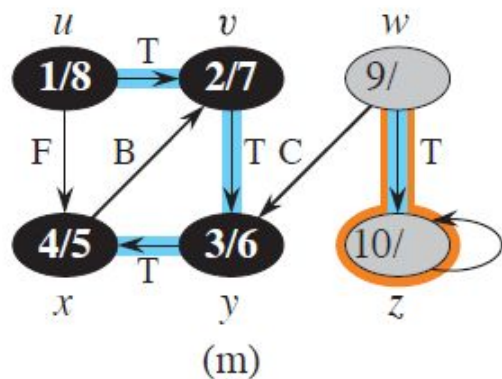
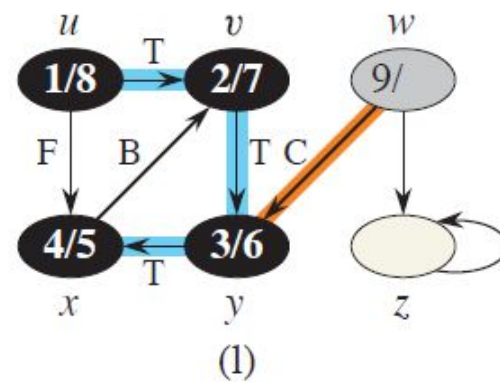
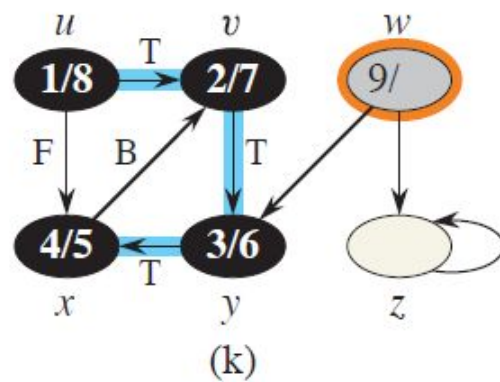
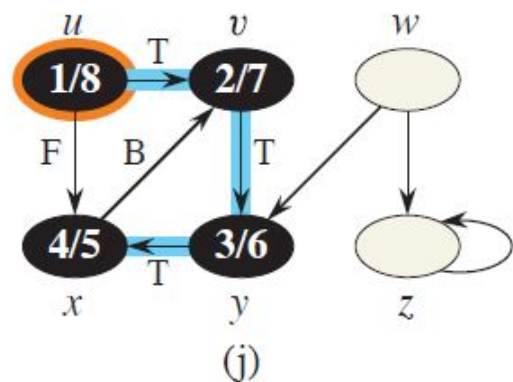
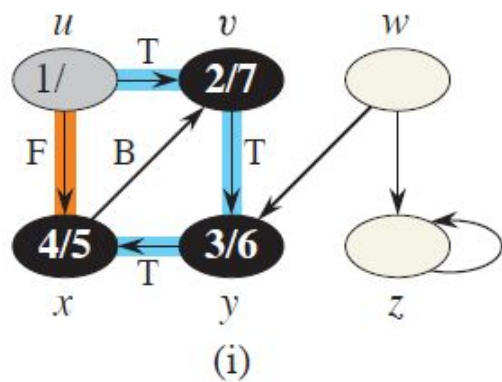


(g)



(h)

DFS Example



DFS Analysis

DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each vertex  $v$  in  $G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $time = time + 1$ 
9   $u.f = time$ 
10  $u.color = \text{BLACK}$ 
```

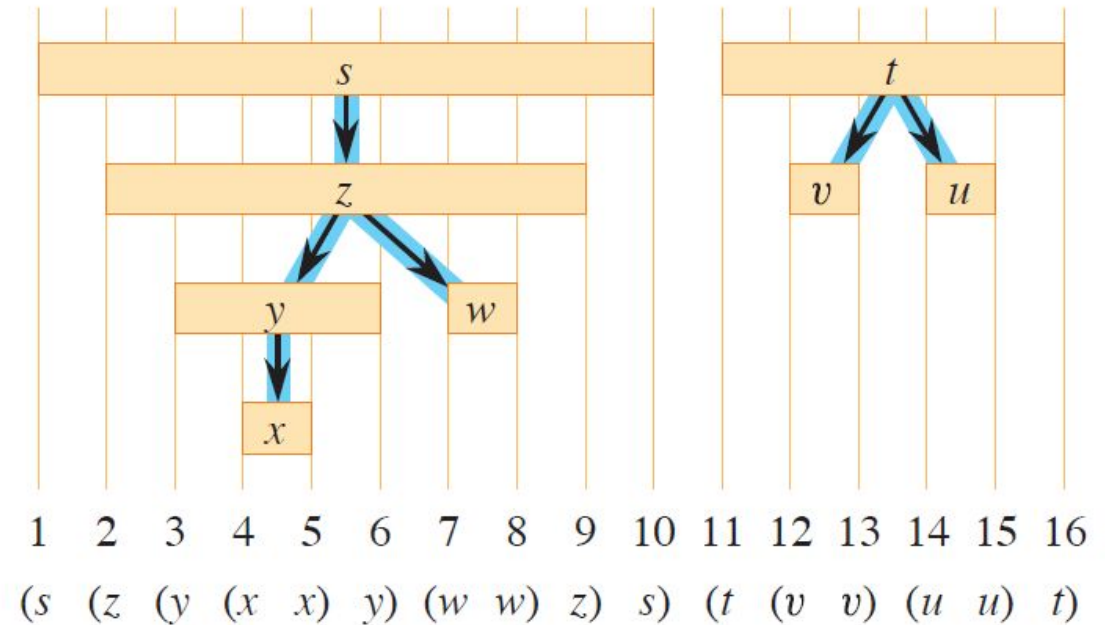
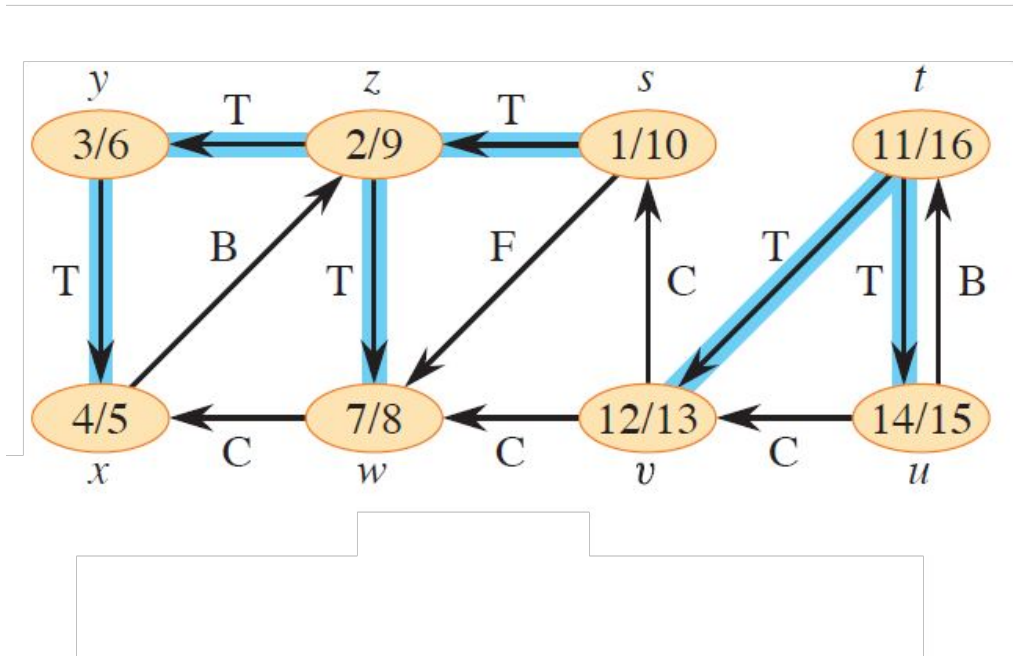
1. Lines 1-3 and 5-7:
 $\theta(V)$ time – excluding visit
2. Only one call of the
DFS_VISIT for each vertex
3. Taking into account

$$\sum_{v \in V} |Adj(v)| = \theta(E)$$

4. Total time is $\theta(V + E)$

DFS: Predecessor Subgraph

- $G_\pi = G(V, E_\pi) ; E_\pi = \{(v.\pi, v) : v \in V \text{ and } v.\pi \neq NULL\}$

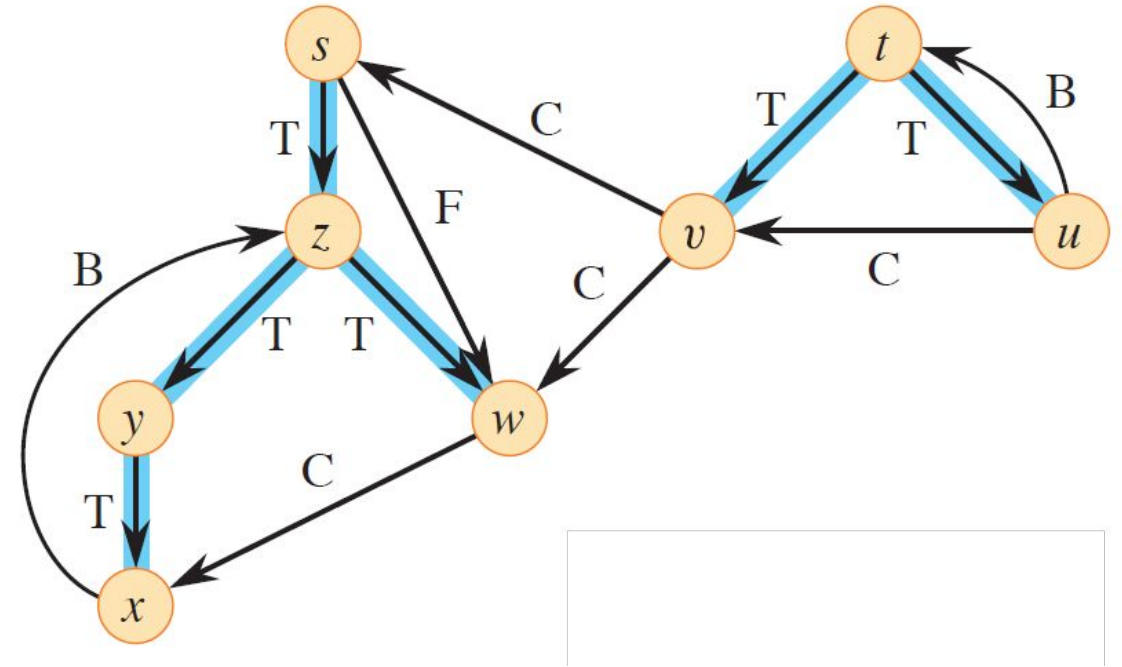


DFS Properties (1)

- The predecessor subgraph G_π does form a forest of trees.
- $u = v.\pi$ if and only if $DFS - VISIT(v)$ was called during a search of u 's adjacency list.
- Discovery and finish times have ***parenthesis structure***
- In any DFS of a (directed or undirected) graph $G(V, E)$, for any two vertices u and v , exactly one of the following three conditions holds
 1. the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest
 2. the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$ and u is a descendant of v in a depth-first tree
 3. the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$ and v is a descendant of u in a depth-first tree

DFS: Classification of Edges

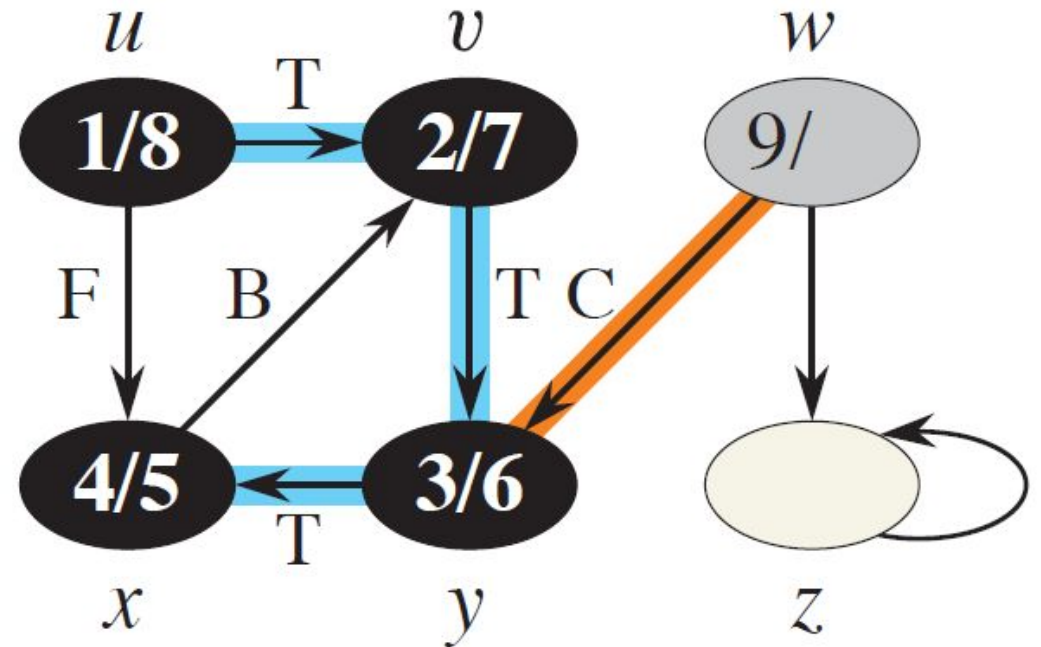
1. **Tree edges** are edges in the Depth-First forest G_π .
2. **Back edges** are those edges (u, v) connecting u to an ancestor v .
3. **Forward edges** are those nontree edges (u, v) connecting a vertex u to a proper descendant v in a depth-first tree.
4. **Cross edges** are all other edges.



DFS: Discovering Edges

When an edge (u, v) is first explored, the color of vertex v says something about the edge:

1. WHITE indicates a tree edge,
2. GRAY indicates a back edge,
3. BLACK indicates a forward or cross edge.



DFS: Edge Classification for Undirected Graph

Theorem: In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Proof: Let (u, v) be an edge of G and $u.d < v.d$

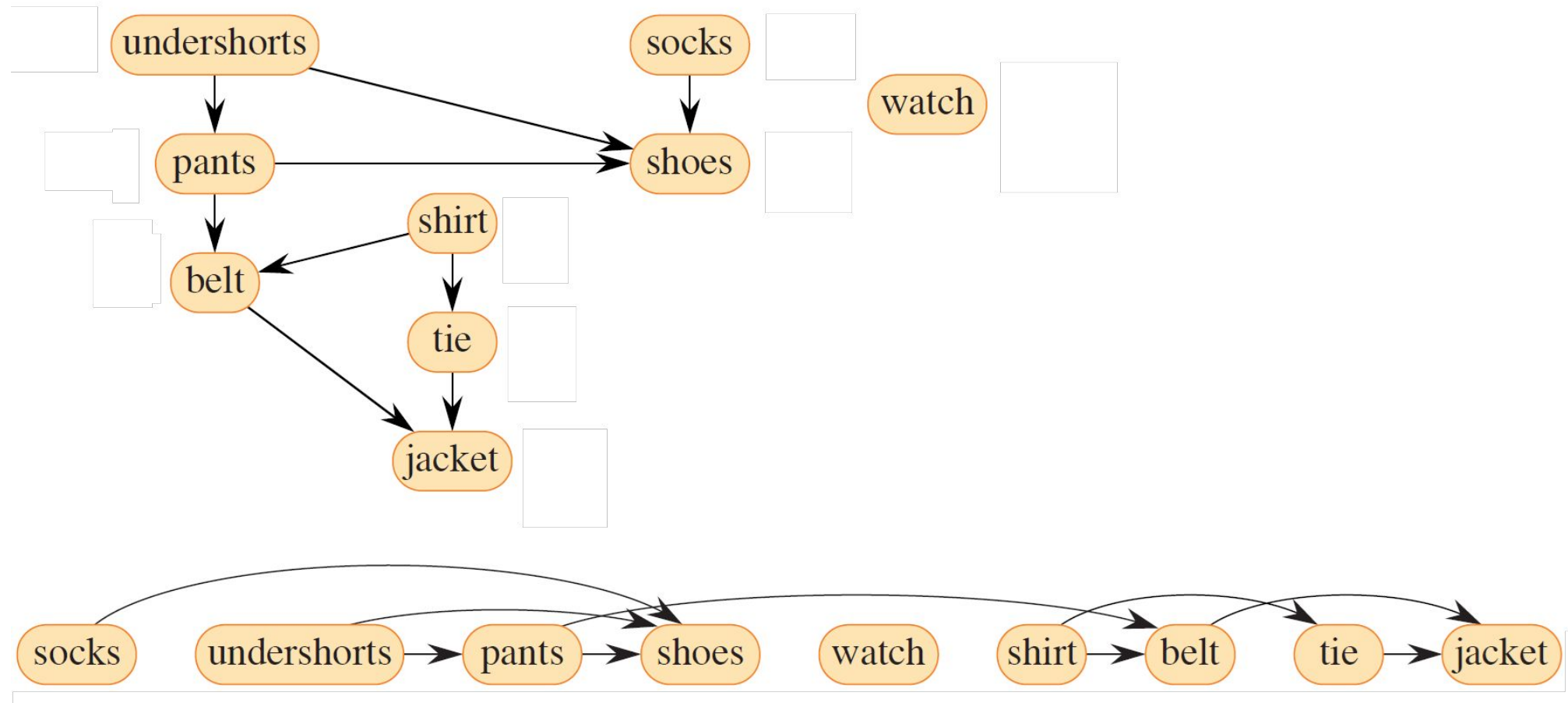
- While u is gray, DFS must discover and finish v before it finishes u .
- If (u, v) is explored from u to v , then $v.d$ was white and (u, v) is a tree edge
- If (u, v) is explored from v to u , then (u, v) is a back edge

Topological Sort

-

Topological sort of a directed acyclic graph (dag) $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge (u, v) then u appears before v in the ordering.

Topological Sort: Example 1



Topological Sort: Pseudocode and Analysis

TOPOLOGICAL-SORT(G)

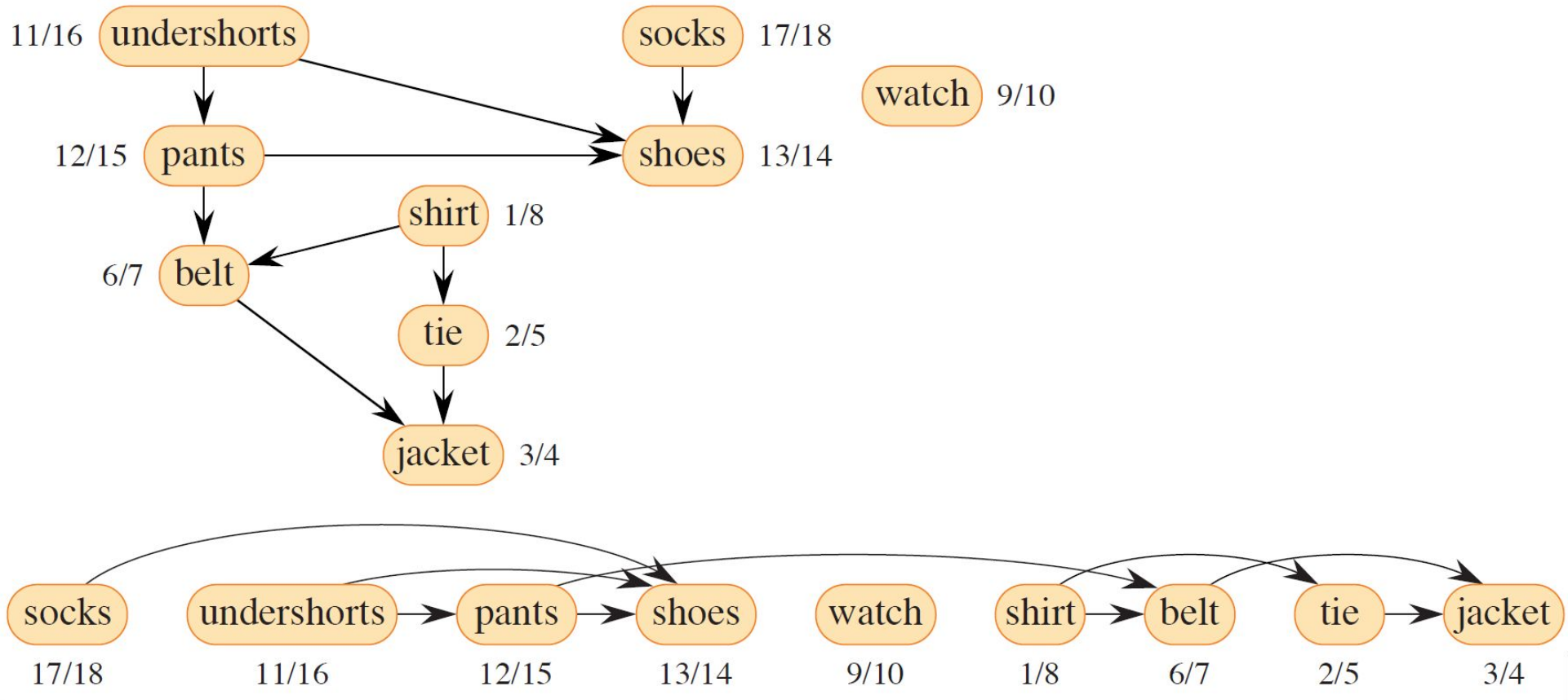
- 1 call DFS(G) to compute finish times $v.f$ for each vertex v
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices

DFS requires $\theta(V + E)$

Inserting onto the front of linked list requires $\theta(1)$ time

Total time $\theta(V + E)$

Topological Sort: Example 2



Topological Sort: Correctness (1)

Lemma: A directed graph G is acyclic if and only if a DFS of G yields no back edges.

Proof:

Suppose that there is a back edge (u, v) the DFS produces. Then v is an ancestor of u in the depth-first forest. So, there is a cycle – path in the tree from v to u plus (u, v) edge.

Suppose there is a cycle c . Let v be the first vertex discovered from cycle c and let (u, v) edge belonging to the c . At the time v is discovered the vertices of c form a path of white vertices from v to u . The node u becomes a descendant of v in the depth-first forest. So (u, v) is a back edge.

Topological Sort: Correctness (2)

Theorem: *TOPOLOGICAL – SORT* produces a topological sort of the directed acyclic graph provided as its input.

Proof:

Suppose that DFS is run on a given dag $G = (V, E)$ to determine finish times for its vertices. It suffices to show that for any pair of distinct vertices $u, v \in V$, if G contains an edge from u to v , then $v.f < u.f$.

When DFS explores (u, v) edge, the v cannot be GRAY.

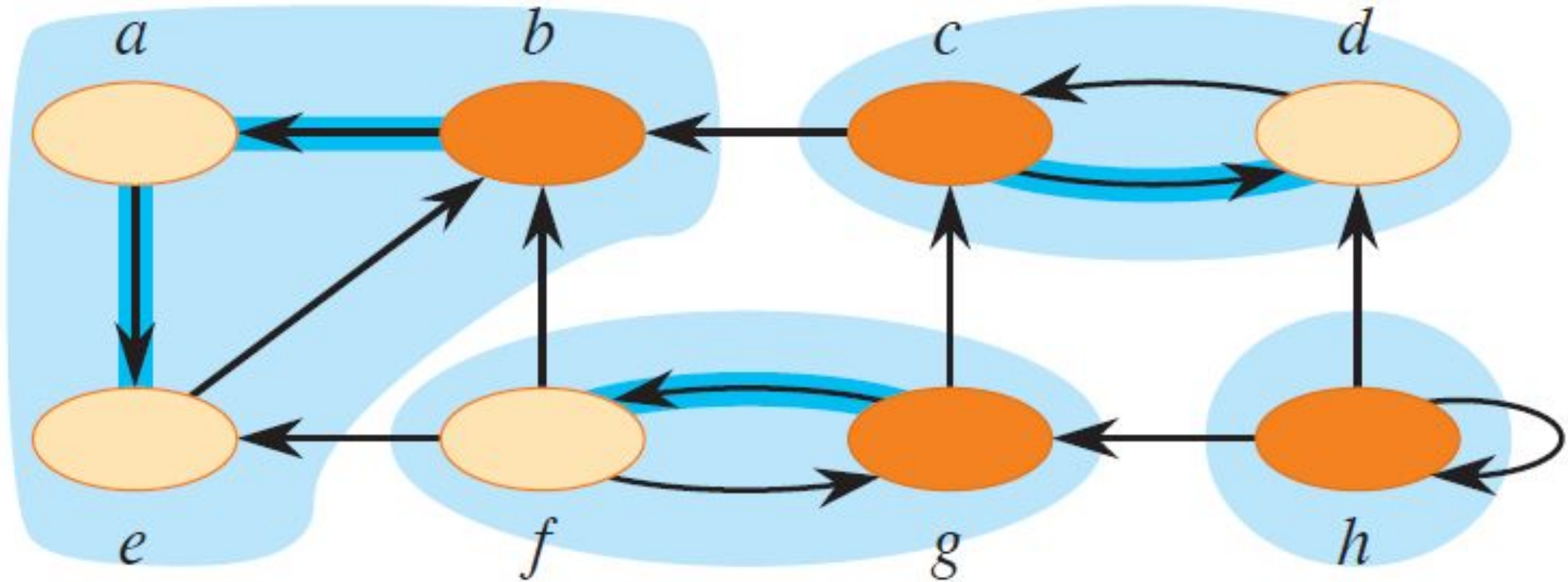
- If v is WHITE, then u is ancestor of v . So, $v.f < u.f$
- if v is BLACK, then v is already finished. So, $v.f < u.f$

Strongly Connected Components

-

Strongly connected component of a directed graph $G = (V, E)$ is a maximal set of vertices $C \subseteq V$ such that for every pair of vertices $u, v \in C$, both $u \rightarrow v$ and $v \rightarrow u$, that is, vertices u and v are reachable from each other.

Strongly Connected Components (SCC)



SCC: Pseudocode and Analysis

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finish times $u.f$ for each vertex u
- 2 create G^T
- 3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

- 1.** $\theta(V + E)$ time required for $DFS(G)$
- 2.** $\theta(V + E)$ time required to create G^T
- 3.** $\theta(V + E)$ time required to create $DFS(G^T)$
- 4.** Total time $\theta(V + E)$



Thank You